



Chapter – 2

jQuery Fundamentals





2.1 Introduction and Basics

jQuery is a JavaScript framework; which purpose is to make it much easier to use JavaScript on your website. You could also describe jQuery as an abstraction layer, since it takes a lot of the functionality that you would have to write many lines of JavaScript to accomplish and wraps it into functions that you can call with a single line of code. It's important to note that jQuery does not replace JavaScript, and while it does offer some syntactical shortcuts, the code you write when you use jQuery is still JavaScript code.

jQuery is a client-side JavaScript library that abstracts away browsers' different implementations into an easy-to-use API. What jQuery does best is to interact with the DOM (add, modify, remove elements on your page), do AJAX requests, create effects (animations) and so forth. It does not provide an application framework, it's merely a tool amongst others that should be used what it's meant to be used for. However, there's a plethora of plug-ins due to a thriving community, and there's pretty much a plug-ins for anything you can think of.

With that in mind, you should be aware that you don't need to be a JavaScript expert to use jQuery. In fact, jQuery tries to simplify a lot of the complicated things from JavaScript, like AJAX calls and DOM manipulation, so that you may do these things without knowing a lot about JavaScript.

There are a bunch of other JavaScript frameworks out there, but as of right now, jQuery seems to be the most popular and also the most extendable, proved by the fact that you can find jQuery plug-ins for almost any task out there. The power, the wide range of plug-ins and the beautiful syntax is what makes jQuery such a great framework. Keep reading to know much more about it and to see why we recommend it.

JQUERY

jQuery is a fast and concise JavaScript Library created by **John Resig** in 2006 with a nice motto – **Write less, do more**. jQuery simplifies HTML document traversing, event handling, animating, and Ajax interactions for rapid web development. jQuery is a JavaScript toolkit designed to simplify various tasks by writing less code.

FEATURES OF JQUERY

- **DOM manipulation** – The jQuery made it easy to select DOM elements, traverse them and modifying their content by using cross-browser open source selector engine called **Sizzle**.
- **Event handling** – The jQuery offers an elegant way to capture a wide variety of events, such as a user clicking on a link, without the need to clutter the HTML code itself with event handlers.
- **AJAX Support** – The jQuery helps you a lot to develop a responsive and feature-rich site using AJAX technology.



- **Animations** – The jQuery comes with plenty of built-in animation effects which you can use in your websites.
- **Lightweight** – The jQuery is very lightweight library - about 19KB in size Minified and gzipped.
- **Cross Browser Support** – The jQuery has cross-browser support, and works well in IE 6.0+, FF 2.0+, Safari 3.0+, Chrome and Opera 9.0+
- **Compatibility All Dynamic Languages** - jQuery script can be use with all most Dynamic Web Languages like PHP, ASP, JSP, CGI etc.
- **Latest Technology** – The jQuery supports CSS3 selectors and basic XPath syntax.

HOW TO USE JQUERY?

There are two ways to use jQuery.

- **Local Installation** – You can download jQuery library on your local machine and include it in your HTML code.
- **CDN Based Version** – You can include jQuery library into your HTML code directly from Content Delivery Network CDN.

LOCAL INSTALLATION

- Go to the <https://jquery.com/download/> to download the latest version available.
- Now put downloaded jquery-2.1.3.min.js file in a directory of your website, e.g. /jquery.

Example

Now you can include jquery library in your HTML file as follows –

```
<html>
<head>
<title>The jQuery Example</title>
<script type = "text/javascript" src = "/jquery/jquery-2.1.3.min.js"></script>
  <script type = "text/javascript">
    $(document).ready(function()
    {
      document.write("Hello, World!");
    });
  </script>
</head>
<body>
  <h1>Hello</h1>
</body>
</html>
```



CDN BASED VERSION

You can include jQuery library into your HTML code directly from Content Delivery Network CDN. Google and Microsoft provides content deliver for the latest version.

Example

Now let us rewrite above example using jQuery library from Google CDN.

```
<html>
<head>
  <title>The jQuery Example</title>
  <script type = "text/javascript"
    src = "http://ajax.googleapis.com/ajax/libs/jquery/2.1.3/jquery.min.js"></script>
  <script type = "text/javascript">
    $(document).ready(function(){
      document.write("Hello, World!");
    });
  </script>
</head>
<body>
  <h1>Hello</h1>
</body>
</html>
```

INSTALLATION OF JQUERY

First of all we have to need jQuery library file. Download latest version of **jquery.js** file from www.jquery.com Website.

Current version is 1.8.0 is Select the **Production (32KB, Minified and Gzipped)** and click on Download [jquery-1.8.0.min.js](http://www.jquery.com/jquery-1.8.0.min.js). Following is the list of available jQuery versions on the site

- Download jquery-1.8.0.min.js (Current Version)
- Download jquery-1.7.2.min.js
- Download jquery-1.7.1.min.js
- Download jquery-1.6.1.min.js
- Download jquery-1.6.min.js
- Download jquery-1.5.2.min.js
- Download jquery-1.5.1.min.js
- Download jquery-1.5.min.js
- Download jquery-1.4.4.min.js



You can rename your file to **jquery.js** for simplicity and put it on the root directory of your website. Following is a small example of using it.

```
<html>
<head>
<title>The jQuery Structure</title>
<script type="text/javascript" src="jquery.js"></script>
  <script type="text/javascript">
    /* add needed javascript code */
  </script>
</head>
<body>
<!-- Some HTML code may be written here -->
</body>
</html>
```

HOW TO CALL A JQUERY LIBRARY FUNCTIONS?

As almost everything we do when using jQuery reads or manipulates the document object model (DOM), we need to make sure that we start adding events etc. as soon as the DOM is ready.

If you want an event to work on your page, you should call it inside the `$(document).ready()` function. Everything inside it will load as soon as the DOM is loaded and before the page contents are loaded.

To do this, we register a ready event for the document as follows –

```
$(document).ready(function() {
  // do stuff when DOM is ready
});
```

To call upon any jQuery library function, use HTML script tags as shown below –

```
<html>
<head>
  <title>The jQuery Example</title>
  <script type = "text/javascript"
    src = "http://ajax.googleapis.com/ajax/libs/jquery/2.1.3/jquery.min.js"></script>
  <script type = "text/javascript" language = "javascript">
    $(document).ready(function() {
      $("div").click(function() {alert("Hello, world!");});
    });
  </script>
</head>
<body>
  <div id = "mydiv">
    Click on this to see a dialogue box.
  </div>
</body>
</html>
```



HOW TO INCLUDE EXTERNAL SCRIPT USING JQUERY?

It is very good way to write the script in external page and then after linked in the document. Its main benefit is reducing complex coding and easy to understand and another benefit is external script is used on one or more documents to reduce the size of the main document.

```
/* Save this Filename: external_demo.js */
$(document).ready(function()
{
    $("div").click(function()
    {
        alert("This Example is External jQuery..!");
    });
});
```

Now we can include external_demo.js file in our main html document.

```
<html>
<head>
<title>The External jQuery Example</title>
<script type="text/javascript"
src="../jquery.min.js"></script>
<script type="text/javascript"
src="external_demo.js"></script>
</head>
<body>
<div id="ex1">
Click here to open Dialogue Box.
</div>
</body>
</html>
```

2.1.1 Advantages of jQuery and Syntax

Advantages

- **Easy to learn:** jQuery is easy to learn because it supports the same JavaScript style coding.
- **Write less do more:** jQuery provides a rich set of features that increase developers' productivity by writing less and readable code.
- **Excellent API Documentation:** jQuery provides excellent online API documentation.
- **Cross-browser support:** jQuery provides excellent cross-browser support without writing extra code.
- **Unobtrusive:** jQuery is unobtrusive which allows separation of concerns by separating HTML and jQuery code.

Syntax

jQuery syntax is made by using HTML elements selector and perform some action on the elements are manipulation in Dot sign (.).



jQuery syntax: **\$(selector).action()**

- \$ sign define the jQuery,
- Selector define the Query Elements in HTML document, and
- action() define the action performed on the elements.

JQUERY BASIC SYNTAX EXAMPLES

\$("#p").hide()

The jQuery hide() function, hiding all <p> elements.

Code	Output
<pre><html> <head> <script type="text/javascript" src="jquery.js"> </script> <script type="text/javascript"> \$(document).ready(function() { \$("#button").click(function() { \$("#p").hide(); }); }); </script> </head> <body> <p>This is a First Paragraph.</p> <p>This is a Second Paragraph.</p> <button>Click Me to Hide Above All Paragraph</button> </body> </html></pre>	This is a First Paragraph. This is a Second Paragraph. Click Me to Hide Above All Paragraph

\$("#this").hide()

The jQuery hide() function, hiding current(this) element.

Code	Output
<pre><html> <head> <script type="text/javascript" src="jquery.js"> </script> <script type="text/javascript"> \$(document).ready(function() { \$("#button").click(function()</pre>	Click Me to Hide THIS button



<pre>{ \$(this).hide(); }); }); </script> </head> <body> <button>Click Me to Hide THIS button</button> </body> </html></pre>	
---	--

\$("#div1").hide()

The jQuery hide() function, hiding whose id="div1" in the elements.

Code	Output
<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function() { \$("button").click(function() { \$("#div1").hide(); }); }); </script> </head> <body> <p id="div1">This is a Second Paragraph.</p> <button>Click Me to Hide Above Paragraph</button> </body> </html></pre>	

\$(".div1").hide()

The jQuery hide() function, hiding whose class=".div1" in the elements.

Code	Output
------	--------



<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function() { \$("button").click(function() { \$(".div1").hide(); }); }); </script> </head> <body> <p>This is a First Paragraph.</p> <p class="div1">This is a Second Paragraph.</p> <button>Click Me to Hide Above Paragraph</button> </body> </html></pre>	This is a First Paragraph. This is a Second Paragraph. Click Me to Hide Above Paragraph
---	--

2.1.2 jQuery: Selectors

jQuery selectors is most important aspects of the jQuery library. jQuery library allows you to select elements in your HTML document by wrapping them in \$(" ") (also you have to use single quotes), which is the jQuery wrapper. Selectors are useful and required at every step while using jQuery. With jQuery selectors you can find elements based on their id, classes, types, attributes, values of attributes and much more. It's based on the existing CSS Selectors, and in addition, it has some own custom selectors.

All type of selectors in jQuery, start with the dollar sign and parentheses: \$().

jQuery Selector Syntax

Selector	Description
TagName / element	Selects all element match of given elements.
This	Selects current elements.
#ID	Selects element whose id is match of given elements.
.CLASS	Selects element whose class is match of given elements.
*	Selects all elements in the document.

ELEMENT / TAG SELECTOR

The jQuery element selector selects elements based on their tag names. You can select all <table> elements on a page like this: \$("table").



Code	Output
<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function() { \$("table *").css("background-color","pink"); }); </script> </head> <body> <table border="0" width="400px"> <tr> <td>Cell 1</td> <td>Cell 2</td> </tr> <tr> <td>Cell 3</td> <td>Cell 4</td> </tr> </table> </body> </html></pre>	

THIS SELECTOR

The jQuery this selector is used for selecting the current element.

Code	Output
<pre><html> <head> <script type="text/javascript" src="jquery.js"> </script> <script type="text/javascript"> \$(document).ready(function() { \$("button").click(function() { \$(this).hide(); }); }); </script> </head></pre>	



<pre><body> <button>Click Me to Hide THIS button</button> </body> </html></pre>	
---	--

ID SELECTOR

The jQuery #id selector uses the id attribute of an HTML tag to find the specific element. An id should be unique within a page, so you should use the #id selector when you want to find a single, unique element.

Code	Output
<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#name").css("background-color","pink"); }); </script> </head> <body> <table border="0" width="400px"> <tr> <td id="name">Cell 1</td> <td>Cell 2</td> </tr> <tr> <td>Cell 3</td> <td>Cell 4</td> </tr> </table> </body> </html></pre>	

CLASS SELECTOR

The jQuery class selector finds elements with a specific class. To find elements with a specific class, write a period character, followed by the name of the class: \$(".test")

Code	Output
------	--------



<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$(".name").css("background-color","pink"); }); </script> </head> <body> <table border="0" width="400px"> <tr> <td class="name">Cell 1</td> <td>Cell 2</td> </tr> <tr> <td>Cell 3</td> <td>Cell 4</td> </tr> </table> </body> </html></pre>	
---	--

jQuery Custom Selectors

Syntax	Description
<u>\$(".:animated")</u>	Selects elements currently being animated.
<u>\$(".:button")</u>	Selects any button elements (inputs or buttons tag).
<u>\$(".:radio")</u>	Selects radio buttons.
<u>\$(".:checkbox")</u>	Selects checkboxes.
<u>\$(".:header")</u>	Selects header elements (h1, h2, h3, etc..).

jQuery Selector References

Following some jQuery Selector References...

Selector	Example	Description
*	<u>\$("table *")</u>	Select All Elements.
#id	<u>\$("#name")</u>	Selected element with id="name".
.class	<u>\$(".name")</u>	Selected elements with class="name".
Tag	<u>\$("p")</u>	Selected All p elements.
Selector	Example	Description
:first	<u>\$("p:first")</u>	first <p> element.



:last	<code>\$("p:last")</code>	last <p> element.
:even	<code>\$("p:even")</code>	Perform all even <tr> elements.
:odd	<code>\$("p:odd")</code>	Perform all odd <tr> elements.
Selector	Example	Description
:enabled	<code>\$(":enabled")</code>	All enabled elements.
:disabled	<code>\$(":disabled")</code>	All disabled elements.
:selected	<code>\$(":selected")</code>	All selected elements.
:checked	<code>\$(":checked")</code>	All checked elements.
Selector	Example	Description
:input	<code>\$(":input")</code>	selected All input elements.
:text	<code>\$(":text")</code>	selected All input elements with type="text".
:button	<code>\$(":button")</code>	selected All input elements with type="button".
:password	<code>\$(":password")</code>	selected All input elements with type="password".
:radio	<code>\$(":radio")</code>	selected All input elements with type="radio".
:checkbox	<code>\$(":checkbox")</code>	selected All input elements with type="checkbox".
:image	<code>\$(":image")</code>	selected All input elements with type="image".
:file	<code>\$(":file")</code>	selected All input elements with type="file".
:submit	<code>\$(":submit")</code>	selected All input elements with type="submit".
:reset	<code>\$(":reset")</code>	selected All input elements with type="reset".
Selector	Example	Description
:header	<code>\$(":header")</code>	Selected All header elements h1...h6.
:animated	<code>\$(":animated")</code>	Selected All animated elements.
:hidden	<code>\$("p:hidden")</code>	Selected All hidden p elements.
:visible	<code>\$("tr:visible")</code>	Selected All visible table rows.
:empty	<code>\$(":empty")</code>	Selected All elements with no child of the elements.
:contains(text)	<code>\$(":contains('Viral Polishwala'))"</code>	Select All elements which contains is text.
Selector	Example	Description
[attribute]	<code>\$("[href]")</code>	Select All elements with a href attribute.
[attribute\$=value]	<code>\$("a:[href\$=.org]")</code>	Selected elements with a href attribute value ending with ".org".
[attribute=value]	<code>\$("a:[href=#]")</code>	Selected elements with a href attribute value equal to "#".
[attribute!=value]	<code>\$("a:[href!=#]")</code>	Selected elements with a href attribute value not equal to "#".

2.1.3 jQuery Events

JavaScript has its own ability to create interaction with Users. Events is a perform action in Dynamic Web Page.



Following are the examples events –

- A mouse click
- A web page loading
- Taking mouse over an element
- Submitting an HTML form
- A keystroke on your keyboard

EVENT TYPES

The term "fires/fired" is often used with events. Example: "The keypress event is fired, the moment you press a key".

Here are some common DOM events:

S.N. Event Type & Description

- 1 **Blur:** Occurs when the element loses focus.
- 2 **Change:** Occurs when the element changes.
- 3 **Click:** Occurs when a mouse click.
- 4 **Dblclick:** Occurs when a mouse double-click.
- 5 **Error:** Occurs when there is an error in loading or unloading etc.
- 6 **Focus:** Occurs when the element gets focus.
- 7 **Keydown:** Occurs when key is pressed.
- 8 **Keypress:** Occurs when key is pressed and released.
- 9 **KeyUp:** Occurs when key is released.
- 10 **Load:** Occurs when document is loaded.
- 11 **Mousedown:** Occurs when mouse button is pressed.
- 12 **Mouseenter:** Occurs when mouse enters in an element region.
- 13 **Mouseleave:** Occurs when mouse leaves an element region.



- 14 **Mousemove:** Occurs when mouse pointer moves.
- 15 **Mouseout:** Occurs when mouse pointer moves out of an element.
- 16 **Mouseover:** Occurs when mouse pointer moves over an element.
- 17 **Mouseup:** Occurs when mouse button is released.
- 18 **Resize:** Occurs when window is resized.
- 19 **Scroll:** Occurs when window is scrolled.
- 20 **Select:** Occurs when a text is selected.
- 21 **Submit:** Occurs when form is submitted.
- 22 **Unload:** Occurs when documents is unloaded.

THE EVENT OBJECT

When these events are triggered you can then use a custom function to do pretty much whatever you want with the event. These custom functions call Event Handlers.

The callback function takes a single parameter; when the handler is called the JavaScript event object will be passed through it.

The event object is often unnecessary and the parameter is omitted, as sufficient context is usually available when the handler is bound to know exactly what needs to be done when the handler is triggered, however there are certain attributes which you would need to be accessed.

THE EVENT ATTRIBUTES

The following event properties/attributes are available and safe to access in a platform independent manner –

Property	Description
altKey	Set to true if the Alt key was pressed when the event was triggered, false if not. The Alt key is labeled Option on most Mac keyboards.
ctrlKey	Set to true if the Ctrl key was pressed when the event was triggered, false if not.
Data	The value, if any, passed as the second parameter to the bind command when the handler was established.
keyCode	For keyup and keydown events, this returns the key that was pressed.
metaKey	Set to true if the Meta key was pressed when the event was triggered, false if not. The Meta key is the Ctrl key on PCs and the Command key on Macs.



pageX	For mouse events, specifies the horizontal coordinate of the event relative from the page origin.
pageY	For mouse events, specifies the vertical coordinate of the event relative from the page origin.
relatedTarget	For some mouse events, identifies the element that the cursor left or entered when the event was triggered.
Screen	For mouse events, specifies the horizontal coordinate of the event relative from the screen origin.
Screen	For mouse events, specifies the vertical coordinate of the event relative from the screen origin.
shiftKey	Set to true if the Shift key was pressed when the event was triggered, false if not.
Target	Identifies the element for which the event was triggered.
Timestamp	The timestamp in milliseconds when the event was created.
Type	For all events, specifies the type of event that was triggered for example, click for example, click.
Which	For keyboard events, specifies the numeric code for the key that caused the event, and for mouse events, specifies which button was pressed 1 for left, 2 for middle, 3 for right

THE EVENT METHODS

bind()

Use	Event occur when One or more event handlers attach the selected match elements.
Syntax	<code>\$(selector).bind(event,[data],function)</code>
Event	event is Required parameter. event define the one or more events attach to the elements.
Data	data is Optional parameter. data define the addition data pass to the function.
Function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").bind("click",function(){ \$("div").fadeTo("slow",0.20); }); }); </script> </head> <body> <button>Click to Show Bind Event</button> <div style="background:pink;width:100%;height:20%;"> </div> </body></pre>



	</html>
--	---------

blur()

Use	Event occur when element lost focus.
Syntax	<code>\$(selector).blur(function)</code> (Bind a Function to blur event) <code>\$(selector).blur()</code> (Trigger the blur event)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").focus(function(){ \$("input").css("background-color","#FFFF99"); }); \$("input").blur(function(){ \$("input").css("background-color","pink"); }); }); </script> </head> <body> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>

change()

Use	Event occurs when change the elements.
Syntax	<code>\$(selector).change(function)</code> (Bind a Function to change event) <code>\$(selector).change()</code> (Trigger the change event)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").change(function(){ \$(this).css("background-color","pink"); }); }); </script></pre>



	<pre></script> </head> <body> <form action=""> Name: <input type="text" name="name" />
 <input type="submit" name="submit" /> </form> </body> </html></pre>
--	--

click()

Use	Event Occurs when the mouse click.
Syntax	<pre>\$(selector).click(function) (Bind a Function to click event) \$(selector).click() (Trigger the click event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").click(function(){ \$(this).css("background-color","pink"); }); }); </script> </head> <body> <form action=""> Name: <input type="text" name="name" />
 <input type="submit" name="submit" /> </form> </body> </html></pre>

dblclick()

Use	Event Occurs when mouse perform double click.
Syntax	<pre>\$(selector).dblclick(function) (Bind a Function to dblclick event) \$(selector).dblclick() (Trigger the dblclick event)</pre>
Function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head></pre>



	<pre><script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").dblclick(function(){ \$(this).css("background-color","pink"); }); }); </script> </head> <body> <form action=""> Name: <input type="text" name="name" />
 <input type="submit" name="submit" /> </form> </body> </html></pre>
--	--

delegate()

Use	Event Occurs when add one or more event handlers, specific child element of matching elements.
Syntax	<code>\$(selector).delegate(ChildSelector,event,[data],function)</code>
Child Selector	ChildSelector is Required parameter. ChildElement define add one or more childelement add to handle event.
event	event is Required parameter. event define add one or more event add to handle event.
data	data is Optional parameter. data define the addition data pass to the function.
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("div").delegate("button","click",function(){ \$("form").slideToggle(); }); }); </script> </head> <body> <div style="background-color: pink"> <button>Click Delegate Event Run</button> <form action=""></pre>



	<pre>Name: <input type="text" name="name" />
 <input type="submit" name="submit" /> </form> </div> </body> </html></pre>
--	---

die()

Use	Event Occurs when remove all event handler along with live() event.
Syntax	<code>\$(selector).die([event],[function])</code>
event	event is Optional parameter. event define add one or more event add to handle event.
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("p").live("click",function(){ \$(this).slideToggle(); }); \$("button").click(function(){ \$("p").die(); }); }); </script> </head> <body> <p>This Me to Shoe Live or Die Event Example</p> <p>This Me and occur slideToogle effect with die or live event</p> <p>This Me to Shoe Live or Die Event Example</p> <button>Click to Run Die Event</button> </body> </html></pre>

error()

Use	Event error Occurs when selected element not loaded successfully.
Syntax	<code>\$(selector).error(function)</code> (Bind a Function to error event) <code>\$(selector).error()</code> (Trigger the error event)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script></pre>



	<pre> <script type="text/javascript"> \$(document).ready(function(){ \$("img").error(function(){ \$("img").replaceWith("Error Loading Image Not Open..!"); }); }); </script> </head> <body> <p>error event occur, when image is not open</p>
 </body> </html> </pre>
--	--

e.pageX()

Use	Event Occurs when mouse position in Left Side.
Syntax	event.pageX
event	event is Required parameter. event define specific event binding to the function.
Example	<pre> <html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$(document).mousemove(function(e){ \$("span").text("X: " + e.pageX + ", Y: " + e.pageY); }); }); </script> </head> <body> <p>Mouse Position is: </p> </body> </html> </pre>

e.pageY()

Use	Event Occurs when mouse position in Top Side.
Syntax	event.pageY
event	event is Required parameter. event define specific event binding to the function.
Example	<pre> <html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> </pre>



	<pre>\$(document).ready(function(){ \$(document).mousemove(function(e){ \$("span").text("X: " + e.pageX + ", Y: " + e.pageY); }); }); </script> </head> <body> <p>Mouse Position is: </p> </body> </html></pre>
--	---

e.timestamp()

Use	Event Occurs on content the number of stored milliseconds since Jan 1, 1970 to current time.
Syntax	event.timeStamp
event	event is Required parameter. event define specific event binding to the function.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(e){ \$("span").text(e.timeStamp); }); }); </script> </head> <body> <p>The click event for the button below occurred <u>_____</u> milliseconds after January 1, 1970.</p> <button>Click to Update MilliSeconds</button> </body> </html></pre>

e.which()

Use	Event Occurs, when key press on your keyboard and return key number.
Syntax	event.which
event	event is Required parameter. event define specific event binding to the function.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script></pre>



	<pre><script type="text/javascript"> \$(document).ready(function(){ \$("input").keydown(function(e){ \$("span").text("Pressed Key: " + e.which); }); }); </script> </head> <body> <p>press key in textbox to show which number of key is press in your keyboard</p> <input type="text" name="inputtext">
 </body> </html></pre>
--	---

focus()

Use	Event Occurs when the element gets focus.
Syntax	<pre>\$(selector).focus(function) (Bind a Function to focus event) \$(selector).focus() (Trigger the focus event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").focus(function(){ \$("input").css("background-color","#FFFF99"); }); \$("input").blur(function(){ \$("input").css("background-color","pink"); }); \$("#btn1").click(function(){ \$("input").focus(); }); \$("#btn2").click(function(){ \$("input").blur(); }); }); </script> </head> <body> <button id="btn1">Click Trigger Focus Event</button>
</pre>



	<pre> <button id="btn2">Click Trigger Blur Event</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html> </pre>
--	---

focusin()

Use	Event Occurs when the element gets focus.
Syntax	<code>\$(selector).focusin(function())</code>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre> <html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("div").focusin(function(){ \$(this).css("background-color","pink"); }); \$("div").focusout(function(){ \$(this).css("background-color","#CCCCCC"); }); }); </script> </head> <body> <p>Click textbox to occur focusin or focusout event</p> <div style="padding:5px;"> <form action=""> Name: <input type="text" name="name" /> </form> </div> </body> </html> </pre>

focusout()

Use	Event Occurs when the element focus out.
Syntax	<code>\$(selector).focusout(function())</code>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre> <html> <head> </pre>



	<pre><script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("div").focusin(function(){ \$(this).css("background-color","pink"); }); \$("div").focusout(function(){ \$(this).css("background-color","#CCCCCC"); }); }); </script> </head> <body> <p>Click textbox to occur focusin or focusout event</p> <div style="padding:5px;"> <form action=""> Name: <input type="text" name="name" /> </form> </div> </body> </html></pre>
--	--

hover()

Use	Event Occurs when the hover on the selected element.
Syntax	<code>\$(selector).hover(FocusIn,FocusOut)</code>
focusIn	FocusIn is Required parameter. FocusIn define mouse in the selected element.
focusOut	FocusOut is Required parameter. FocusOut define mouse out the selected element.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("div").hover(function(){ \$("div").css("background-color","pink"); },function(){ \$("div").css("background-color","#CCCCCC"); }); }); </script> </head> <body> <p>Click textbox to occur focusin or focusout event</p></pre>



	<pre><div style="padding:5px;"> <form action=""> Name: <input type="text" name="name" /> </form> </div> </body> </html></pre>
--	---

keydown()

Use	Event Occurs when key is pressed.
Syntax	<pre>\$(selector).keydown(function) (Bind a Function to keydown event) \$(selector).keydown() (Trigger the keydown event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").keydown(function(){ \$("input").css("background-color","#FFFF99"); }); \$("input").keyup(function(){ \$("input").css("background-color","pink"); }); \$("#btn1").click(function(){ \$("input").keydown(); }); \$("#btn2").click(function(){ \$("input").keyup(); }); }); </script> </head> <body> <button id="btn1">Function keyDown Event Run</button> <button id="btn2">Function keyUp Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>



keypress()

Use	Event Occurs when key is pressed count the pressed key.
Syntax	<code>\$(selector).keypress(function)</code> (Bind a Function to keypress event) <code>\$(selector).keypress()</code> (Trigger the keypress event)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> i=0; \$(document).ready(function(){ \$("input").keypress(function(){ \$("span").text(i=i+1); }); \$("button").click(function(){ \$("input").keypress(); }); }); </script> </head> <body> <button>Function keypress Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> <p>No of Keypressed: </p> </body> </html></pre>

keyup()

Use	Occurs when key is released.
Syntax	<code>\$(selector).keyup(function)</code> (Bind a Function to keyup event) <code>\$(selector).keyup()</code> (Trigger the keyup event)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").keydown(function(){ \$("input").css("background-color", "#FFFF99"); }); }); </script> </head> <body> <input type="text" /> </body> </html></pre>



	<pre>}); \$("input").keyup(function(){ \$("input").css("background-color","pink"); }); \$("#btn1").click(function(){ \$("input").keydown(); }); \$("#btn2").click(function(){ \$("input").keyup(); }); }); </script> </head> <body> <button id="btn1">Function keyDown Event Run</button> <button id="btn2">Function keyUp Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>
--	--

live()

Use	Event Occurs when live all event handler.
Syntax	<code>\$(selector).live([event],[function])</code>
event	event is Optional parameter. event define add one or more event add to handle event.
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("p").live("click",function(){ \$(this).slideToggle(); }); \$("button").click(function(){ \$("p").die(); }); }); </script> </head> <body></pre>



	<pre><p>This Me to Shoe Live or Die Event Example</p> <p>This Me and occur slideToogle effect with die or live event</p> <p>This Me to Shoe Live or Die Event Example</p> <button>Click to Run Die Event</button> </body> </html></pre>
--	---

load()

Use	Event occurs when document is load.
Syntax	<code>\$(selector).load(function)</code>
function	Function is Required parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("img").load(function(){ \$("div").text("Current Load Event is Run"); }); }); </script> </head> <body> <div>Image is Loading</div> </body> </html></pre>

mousedown()

Use	Event Occurs when mouse button is pressed.
Syntax	<code>\$(selector).mousedown(function)</code> (Bind Function to mousedown) <code>\$(selector).mousedown()</code> (Trigger the mousedown event)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").mousedown(function(){ \$("input").css("background-color", "#FFFF99"); }); \$("input").mouseup(function(){ \$("input").css("background-color", "pink"); }); }); </script> </head> <body> <input type="text" value="Click to Run Die Event" /> </body> </html></pre>



	<pre>}); \$("#btn1").click(function(){ \$("input").mousedown(); }); \$("#btn2").click(function(){ \$("input").mouseup(); }); }); </script> </head> <body> <button id="btn1">Function MouseDown Event Run</button> <button id="btn2">Function MouseUp Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>
--	--

mouseenter()

Use	Event Occurs when mouse enters in an element area.
Syntax	<pre>\$(selector).mouseenter(function) (Bind Function to mouseenter) \$(selector).mouseenter() (Trigger the mouseenter event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").mouseenter(function(){ \$("input").css("background-color","#FFFF99"); }); \$("input").mouseleave(function(){ \$("input").css("background-color","pink"); }); \$("#btn1").click(function(){ \$("input").mouseenter(); }); \$("#btn2").click(function(){ \$("input").mouseleave(); }); });</pre>



	<pre></script> </head> <body> <button id="btn1">Function MouseEnter Event Run</button> <button id="btn2">Function MouseLeave Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>
--	--

mouseleave()

Use	Event Occurs when mouse leaves an element area.
Syntax	<pre>\$(selector).mouseleave(function) (Bind Function to mouseleave) \$(selector).mouseleave() (Trigger the mouseleave event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").mouseenter(function(){ \$("input").css("background-color","#FFFF99"); }); \$("input").mouseleave(function(){ \$("input").css("background-color","pink"); }); \$("#btn1").click(function(){ \$("input").mouseenter(); }); \$("#btn2").click(function(){ \$("input").mouseleave(); }); }); </script> </head> <body> <button id="btn1">Function MouseEnter Event Run</button> <button id="btn2">Function MouseLeave Event Run</button> <form action=""> Name: <input type="text" name="name" /></pre>



	<code></form></code> <code></body></code> <code></html></code>
--	--

mousemove()

Use	Event Occurs when mouse pointer moves.
Syntax	<code>\$(selector).mousemove(function)</code> (Bind Function to mousemove)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$(document).mousemove(function(e){ \$("span").text("X: " + e.pageX + ", Y: " + e.pageY); }); }); </script> </head> <body> <p>Mouse Position is: </p> </body> </html></pre>

mouseout()

Use	Event Occurs when mouse pointer out an element.
Syntax	<code>\$(selector).mouseout(function)</code> (Bind Function to mouseout) <code>\$(selector).mouseout()</code> (Trigger the mouseout event)
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").mouseover(function(){ \$("input").css("background-color", "#FFFF99"); }); \$("input").mouseout(function(){ \$("input").css("background-color", "pink"); }); \$("#btn1").click(function(){ \$("input").mouseover(); }); }); </script> </head> <body> <input type="text"/> <input type="button" value="Click Me" id="btn1"/> </body> </html></pre>



	<pre>}); \$("#btn2").click(function(){ \$("input").mouseout(); }); }); </script> </head> <body> <button id="btn1">Function Mouseover Event Run</button> <button id="btn2">Function Mouseout Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>
--	---

mouseover()

Use	Event Occurs when mouse pointer moves over an element.
Syntax	<pre>\$(selector).mouseover(function) (Bind Function to mouseover) \$(selector).mouseover() (Trigger the mouseover event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").mouseover(function(){ \$("input").css("background-color","#FFFF99"); }); \$("input").mouseout(function(){ \$("input").css("background-color","pink"); }); \$("#btn1").click(function(){ \$("input").mouseover(); }); \$("#btn2").click(function(){ \$("input").mouseout(); }); }); </script> </head> <body></pre>



	<pre><button id="btn1">Function Mouseover Event Run</button> <button id="btn2">Function Mouseout Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>
--	--

mouseup()

Use	Event Occurs when mouse button is released.
Syntax	<pre>\$(selector).mouseup(function) (Bind Function to mouseup) \$(selector).mouseup() (Trigger the mouseup event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").mousedown(function(){ \$("input").css("background-color","#FFFF99"); }); \$("input").mouseup(function(){ \$("input").css("background-color","pink"); }); \$("#btn1").click(function(){ \$("input").mousedown(); }); \$("#btn2").click(function(){ \$("input").mouseup(); }); }); </script> </head> <body> <button id="btn1">Function MouseDown Event Run</button> <button id="btn2">Function MouseUp Event Run</button> <form action=""> Name: <input type="text" name="name" /> </form> </body> </html></pre>



ready()

Use	Event Occurs when function is ready to document.
Syntax	<code>\$(document).ready(function) (Bind Function to ready event)</code>
Function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre> <html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("p").hide(); \$("#btn1").click(function () { \$("p").toggle("slow"); }); }); </script> </head> <body> <button id="btn1">Click To Show Paragraph</button> <p style="background-color:#99FFFF;font-size:16px;font-family:Verdana;">This Effect is Toggle Effect with ready event work.</p> </body> </html> </pre>

select()

Use	Event Occurs when a text is selected.
Syntax	<code>\$(selector).select(function) (Bind Function to select event)</code> <code>\$(selector).select() (Trigger the select event)</code>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre> <html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("input").select(function(){ \$("span").text("Text Selected"); }); \$("button").click(function(){ \$("input").select(); }); }); </script> </pre>



	<pre></head> <body> <input type="text" value="Click to Select" />
 <button>Select All Textbox text with Select event.</button> </body> </html></pre>
--	---

submit()

Use	Event Occurs when form is submitted.
Syntax	<pre>\$(selector).submit(function) (Bind Function to submit event) \$(selector).select() (Trigger the submit event)</pre>
function	Function is Optional parameter. Function define the ready to run when event occur.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("form").submit(function(e){ alert("Successfully Submitted"); }); \$("button").click(function(){ \$("form").submit(); }); }); </script> </head> <body> <form action="#" method="get"> Name: <input type="text" name="name" />
 Password: <input type="password" name="password"/>
 <input type="submit" name="submit"/> </form> </body> </html></pre>

unload()

Use	Event Occurs when documents is unloaded.
Syntax	<pre>\$(selector).unload(function)</pre>



function	Function is Required parameter. Function define the ready to run when event occur.
Example	<pre> <html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$(window).unload(function(){ alert("Close this window to show Me Unload Event occur..!"); }); }); </script> </head> <body> <div>Image is Loading</div> </body> </html> </pre>

2.2 jQuery : Effects

jQuery provides a trivially simple interface for doing various kind of amazing effects. jQuery methods allow us to quickly apply commonly used effects with a minimum configuration.

HIDE

Use	The hide method simply hides each of the set of matched elements if they are shown. There is another form of this method which controls the speed of the animation.
Syntax	selector.hide();
Parameter	NA
Example	<pre> <html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#btn1").hide(); \$("p").hide(); \$("#btn2").click(function () { \$("p").show("slow"); \$("#btn2").hide(); \$("#btn1").show(); }); \$("#btn1").click(function () { \$("p").hide("slow"); }); }); </pre>



	<pre>}); </script> </head> <body> <button id="btn1">Click To Hide Paragraph</button> <button id="btn2">Click To Show Paragraph</button> <p style="background-color:#99FFFF;font-size:16px;font-family:Verdana;">This Paragraph Will Be Hide After Click...</p> </body> </html></pre>
--	--

SHOW

Use	The show() method simply shows each of the set of matched elements if they are hidden. There is another form of this method which controls the speed of the animation.
Syntax	selector.show();
Parameter	NA
Example	<pre><html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#btn1").hide(); \$("p").hide(); \$("#btn2").click(function () { \$("p").show("slow"); \$("#btn2").hide(); \$("#btn1").show(); }); \$("#btn1").click(function () { \$("p").hide("slow"); }); }); </script> </head> <body> <button id="btn1">Click To Hide Paragraph</button> <button id="btn2">Click To Show Paragraph</button> <p style="background-color:#99FFFF;font-size:16px;font-family:Verdana;">This Paragraph Will Be Hide After Click...</p> </body> </html></pre>



FADE

Fadein()	
Use	The fadeIn() method fades in all matched elements by adjusting their opacity and firing an optional callback after completion.
Syntax	selector.fadeIn(speed, [callback]);
Parameter	speed – A string representing one of the three predefined speeds "slow","def",or"fast""slow","def",or"fast" or the number of milliseconds to run the animation e.g.1000e.g.1000. callback – This is optional parameter representing a function to call once the animation is complete.
Example	<pre> <html> <head> <script src=" jquery.js " type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#btn1").click(function(){ \$("div").fadeOut(5000); }); \$("#btn2").click(function(){ \$("div").fadeIn(5000); }); }); </script> </head> <body> <div style="background:#FF9933;width:100%;">My Effect is fadeOut Effect</div> <button id="btn1">Fade Out (5 Second)</button> <button id="btn2">Fade In (5 Second)</button> </body> </html> </pre>
Fadeout()	
Use	The fadeOut() method fades out all matched elements by adjusting their opacity to 0, then setting display to "none" and firing an optional callback after completion.
Syntax	selector.fadeOut(speed, [callback]);
Parameter	speed – A string representing one of the three predefined speeds "slow","def",or"fast""slow","def",or"fast" or the number of milliseconds to run the animation e.g.1000e.g.1000. callback – This is optional parameter representing a function to call once the animation is complete.
Example	<pre> <html> <head> </pre>



	<pre><script src=" jquery.js " type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#btn1").click(function(){ \$("div").fadeOut(5000); }); \$("#btn2").click(function(){ \$("div").fadeIn(5000); }); }); </script> </head> <body> <div style="background:#FF9933;width:100%;">My Effect is fadeOut Effect</div> <button id="btn1">Fade Out (5 Second)</button> <button id="btn2">Fade In (5 Second)</button> </body> </html></pre>
Fadeto()	
Use	The fadeto() method fades the opacity of all matched elements to a specified opacity and firing an optional callback after completion.
Syntax	selector.fadeTo(speed, opacity[, callback]);
Parameter	• speed – A string representing one of the three predefined speeds "slow", "def", or "fast" or the number of milliseconds to run the animation e.g. 1000 e.g. 1000. • opacity – A number between 0 and 1 denoting the target opacity. • callback – This is optional parameter representing a function to call once the animation is complete.
Example	<pre><html> <head> <script src=" jquery.js " type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("div").fadeTo("slow",0.20); }); }); </script> </head> <body> <button>Click to Show FadeTo Effect</button> <div style="background:#FF9933;width:100%;height:20%;"> </div></pre>



	<code></body></code> <code></html></code>
--	--

SLIDE

jQuery slide methods use to change element height is visible or hidden in a Selected elements. jQuery Slide Method support main three methods..

slideDown()	
Use	The slideDown() method reveals all matched elements by adjusting their height and firing an optional callback after completion.
Syntax	<code>\$(selector).slideDown(speed,[callback]);</code>
Parameter	• speed: Elements is a represent by three predefined String speed ("slow", "normal", "fast"). otherwise number represent by milliseconds (Ex. 3000). • callback: Callback is optional parameter. It is use to represents a function to be executed whenever effect is completed.
Example	<pre><html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#click").click(function(){ \$("#slide").slideDown("slow"); }); }); </script> <style type="text/css"> #slide, #click { font-family:Verdana; font-size:14px; background:#CCCCCC; border:solid 1px #c3c3c3; text-align:center; } #slide { display:none; height:60px; } </style> </head> <body> <div id="slide"></pre>



	<pre><p>This is a jQuery Effect Example you are really enjoy this to see this effect.
</p> </div> <p id="click">Show Slide Down Panel</p> </body></html></pre>
slideUp()	
Use	The slideUp() method hides all matched elements by adjusting their height and firing an optional callback after completion.
Syntax	<code>\$(selector).slideUp(speed,[callback]);</code>
Parameter	• speed: Elements is a represent by three predefined String speed ("slow", "normal", "fast"). otherwise number represent by milliseconds (Ex. 3000). • callback: Callback is optional parameter. It is use to represents a function to be executed whenever effect is completed.
Example	<pre><html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#click").click(function(){ \$("#slide").slideUp("slow"); }); }); </script> <style type="text/css"> #slide, #click { font-family:Verdana; font-size:14px; background:#CCCCCC; border:solid 1px #c3c3c3; text-align:center; } #slide { height:60px; } </style> </head> <body> <div id="slide"> <p>This is a jQuery Effect Example you are really enjoy this to see this effect.
</p> </div> <p id="click">Show Slide Up Panel</p></pre>



	</body> </html>
slidetoggle()	
Use	The slideToggle() method toggles the visibility of all matched elements by adjusting their height and firing an optional callback after completion.
Syntax	\$(selector).slideToggle(speed,[callback]);
Parameter	• speed: Elements is a represent by three predefined String speed ("slow", "normal", "fast"). otherwise number represent by milliseconds (Ex. 3000). • callback: Callback is optional parameter. It is use to represents a function to be executed whenever effect is completed.
Example	<pre> <html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("#click").click(function(){ \$("#slide").slideToggle("slow"); }); }); </script> <style type="text/css"> #slide, #click { font-family:Verdana; font-size:14px; background:#CCCCCC; border:solid 1px #c3c3c3; text-align:center; } #slide { height:60px; display:none; } </style> </head> <body> <div id="slide"> <p>This is a jQuery Effect Example you are really enjoy this to see this effect.
</p> </div> <p id="click">Show Slide Toogle Panel</p> </body> </html> </pre>



ANIMATE

Use	The animate() method performs a custom animation of a set of CSS properties.
Syntax	selector.animate(params, [duration, easing, callback]);
Parameter	• params – A map of CSS properties that the animation will move toward. • duration – This is optional parameter representing how long the animation will run. • easing – This is optional parameter representing which easing function to use for the transition. • callback – This is optional parameter representing a function to call once the animation is complete.
Example	<pre><html> <head> <script src="jquery.js" type="text/javascript"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("div").animate({fontSize:"60px"},"slow"); }); }); </script> </head> <body style="overflow:auto;"> <button>Start Animation</button> <div style="background:#FF9933;height:75px;position:relative;width:100%;">My Animate Effect</div> </body></html></pre>

STOP

Use	jQuery stop() effect method stop the running animation of selected elements.
Syntax	\$(selector).stop([stopAll])
Parameter	• stopall – stopAll is Optional parameter. A Boolean value specifying stop the queued animate. Default value is false.
Example	<pre><html> <head> <script type="text/javascript" src="jquery-1.8.0.min.js"></script> <script type="text/javascript"> \$(document).ready(function() { \$("#btn1").click(function(){ \$("#divanimate").animate({height:"300px"},8000); }); \$("#btn2").click(function(){ \$("#divanimate").stop(); }); }); </script> </head> <body> <div id="divanimate" style="background-color:blue; height:100px; width:100%; text-align:center; color:white; font-size:24px; line-height:100px;"> Animate Effect </div> <div style="display:flex; justify-content:space-around; margin-top:10px;"> <button id="btn1">Start Animation <button id="btn2">Stop Animation </div> </body> </html></pre>



	<pre>}); }); </script> </head> <body> <button id="btn1">Animate Start</button> <button id="btn2">Stop Efeect</button> <div id="divanimate" style="background: pink;margin:10px;height:100px;width:100px;" ></div> </body> </html></pre>
--	---

CALLBACK AND FUNCTIONS

Use	A callback function is executed after the current effect is 100% finished. JavaScript statements are executed line by line. However, with effects, the next line of code can be run even though the effect is not finished. This can create errors. To prevent this, you can create a callback function. A callback function is executed after the current effect is finished.
Syntax	<code>\$(selector).hide(speed,callback);</code>
Parameter	• speed: Elements is a represent by three predefined String speed ("slow", "normal", "fast"). otherwise number represent by milliseconds (Ex. 3000). • callback: Callback is optional parameter. It is use to represents a function to be executed whenever effect is completed.
Example	<pre><html> <head> <script src=" jquery.js"></script> <script> \$(document).ready(function(){ \$("button").click(function(){ \$("p").hide("slow", function(){ alert("The paragraph is now hidden"); }); }); }); </script> </head> <body> <button>Hide</button> <p>This is a paragraph with little content.</p></body></html></pre>



	<pre><html> <head> <script src=" jquery.js"></script> <script> \$(document).ready(function(){ \$("button").click(function(){ \$("p").hide(1000); alert("The paragraph is now hidden"); }); }); </script> </head> <body> <button>Hide</button> <p>This is a paragraph with little content.</p> </body> </html></pre>
--	---

CHAINING

With jQuery, you can chain together actions/methods. Chaining allows us to run multiple jQuery methods (on the same element) within a single statement.

Until now we have been writing jQuery statements one at a time (one after the other). However, there is a technique called chaining, that allows us to run multiple jQuery commands, one after the other, on the same element(s).

To chain an action, you simply append the action to the previous action. The following example chains together the `css()`, `slideUp()`, and `slideDown()` methods. The "p1" element first changes to red, then it slides up, and then it slides down:

```
<html>
<head>
<script src=" jquery.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#p1").css("color", "red").slideUp(2000).slideDown(2000);
    });
});
</script>
</head>
<body>
<p id="p1">jQuery is fun!!</p>
<button>Click me</button>
```



```
</body>
</html>
```

We could also have added more method calls if needed. When chaining, the line of code could become quite long. However, jQuery is not very strict on the syntax; you can format it like you want, including line breaks and indentations.

```
<html>
<head>
<script src="jquery.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#p1").css("color", "red")
        .slideUp(2000)
        .slideDown(2000);
    });
});
</script>
</head>
<body>
<p id="p1">jQuery is fun!!</p>
<button>Click me</button>
</body>
</html>
```

2.3 jQuery : HTML Manipulation Methods

jQuery HTML Methods are performed for changing and manipulate the HTML elements or attributes.

JQUERY GET

One very important part of jQuery is the possibility to manipulate the DOM (Document Object Model). jQuery comes with a bunch of DOM related methods that make it easy to access and manipulate elements and attributes.

Three simple, but useful, jQuery methods for DOM manipulation are:

- **text()** - Returns the text content of selected elements
- **html()** - Returns the content of selected elements (including HTML markup)
- **val()** - Returns the value of form fields

Example (Get content with HTML and TEXT)

```
<html>
```



```
<head>
<script src=" jquery.js"></script>
<script>
$(document).ready(function(){
    $("#btn1").click(function(){
        alert("Text: " + $("#test").text());
    });
    $("#btn2").click(function(){
        alert("HTML: " + $("#test").html());
    });
});
</script>
</head>
<body>
<p id="test">This is some <b>bold</b> text in a paragraph.</p>
<button id="btn1">Show Text</button>
<button id="btn2">Show HTML</button>
</body>
</html>
```

Example (Manipulating element content)

```
<html>
<head>
<script type="text/javascript" src="jquery.js"></script>
<script type="text/javascript">
$(document).ready(function(){
    $("#button").click(function(){
        $("#p").html("Congratulation, Viral Polishwala");
    });
});
</script>
</head>
<body>
<p>This is a paragraph.</p>
<button>Change content of p elements</button>
</body>
</html>
```

Example (Getting value of input field with val())

```
<html>
<head>
<script src=" jquery. js"></script>
<script>
```



```
$(document).ready(function(){
    $("button").click(function(){
        alert("Value: " + $("#test").val());
    });
});
</script>
</head>
<body>
<p>Name: <input type="text" id="test" value="Mickey Mouse"></p>
<button>Show Value</button>
</body>
</html>
```

Example (Getting value of attribute with attr())

```
<html>
<head>
<script src=" jquery.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        alert($("#vbp").attr("href"));
    });
});
</script>
</head>
<body>
<p>Contact : <a href="http://www.viralpolishwala.co.in" id="vbp">Viral Polishwala</a></p>
<button>Show href Value</button>
</body>
</html>
```

JQUERY SET

We will use the same three methods from the previous page to set content:

- **text()** - Sets the text content of selected elements
- **html()** - Sets the content of selected elements (including HTML markup)
- **val()** - Sets the value of form fields

Example (Use of text(), html() and val() for setting new content)

```
<html>
<head>
<script src=" jquery.js "></script>
```



```
<script>
$(document).ready(function(){
    $("#btn1").click(function(){
        $("#test1").text("Hello world!");
    });
    $("#btn2").click(function(){
        $("#test2").html("<b>Hello world!</b>");
    });
    $("#btn3").click(function(){
        $("#test3").val("Dolly Duck");
    });
});
</script>
</head>
<body>
<p id="test1">This is a paragraph.</p>
<p id="test2">This is another paragraph.</p>
<p>Input field: <input type="text" id="test3" value="Mickey Mouse"></p>
<button id="btn1">Set Text</button>
<button id="btn2">Set HTML</button>
<button id="btn3">Set Value</button>
</body>
</html>
```

All of the three jQuery methods above: `text()`, `html()`, and `val()`, also come with a callback function. The callback function has two parameters: the index of the current element in the list of elements selected and the original (old) value. You then return the string you wish to use as the new value from the function.

Example (Use of `text( )` and `html( )` with a callback function)

```
<html>
<head>
<script src="jquery.js"></script>
<script>
$(document).ready(function(){
    $("#btn1").click(function(){
        $("#test1").text(function(i, origText){
            return "Old text: " + origText + " New text: Hello world! (index: " + i + ")";
        });
    });

    $("#btn2").click(function(){
        $("#test2").html(function(i, origText){
```




```
        return "Old html: " + origText + " New html: Hello <b>world!</b> (index: " + i + ")";
    });
});
</script>
</head>
<body>
<p id="test1">This is a <b>bold</b> paragraph.</p>
<p id="test2">This is another <b>bold</b> paragraph.</p>
<button id="btn1">Show Old/New Text</button>
<button id="btn2">Show Old/New HTML</button>
</body>
</html>
```

The jQuery `attr()` method is also used to set/change attribute values.

Example (Use of `attr()` for setting new value to attribute)

```
<html>
<head>
<script src="jquery.js"></script>
<script>
$(document).ready(function(){
    $("#vbp").click(function(){
        $("#vbp").attr("href", "http://www.viralpolishwala.co.in/downloads");
    });
});
</script>
</head>
<body>
<p><a href="http://www.viralpolishwala.co.in" id="vbp">Viral B. Polishwala</a></p>
<button>Change href Value</button>
<p>Mouse over the link (or click on it) to see that the value of the href attribute has changed.</p>
</body>
</html>
```

JQUERY ADD

With jQuery, it is easy to add new elements/content. We will look at four jQuery methods that are used to add new content:

- **append()** - Inserts content at the end of the selected elements

Use	jQuery <code>append()</code> method Inserts content into inside end of the selected elements.
Syntax	<code>\$(selector).append(content)</code>
Parameter	• Content: content is Required parameter. Insert content end into selected element.



Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").append(" Online Web Development Tutorial"); }); }); </script> </head> <body> <p>Viral Polishwala </p> <button>Insert append end of p element</button> </body></html></pre>
	<pre><html> <head> <script src="jquery.js"></script> <script> function appendText() { var txt1 = "<p>Text.</p>"; // Create text with HTML var txt2 = \$("<p></p>").text("Text."); // Create text with jQuery var txt3 = document.createElement("p"); txt3.innerHTML = "Text."; // Create text with DOM \$("body").append(txt1, txt2, txt3); // Append new elements } </script> </head> <body> <p>This is a paragraph.</p> <button onclick="appendText()">Append text</button> </body> </html></pre>

- **prepend()** - Inserts content at the beginning of the selected elements

Use	jQuery prepend() method Inserts content into inside beggining of the selected elements.
Syntax	\$(selector).prepend(content)
Parameter	• Content: content is Required parameter. Insert content inside first into selected element.



Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").prepend(" Online Web Development Tutorial"); }); }); </script> </head> <body> <p>Viral Polishwala </p> <button>Insert prepend end of p element</button> </body> </html></pre>
----------------	---

- **after()** - Inserts content after the selected elements

Use	jQuery after() method add into end of selected elements.
Syntax	\$(selector).after(content)
Parameter	• Content: content is Required parameter. Insert content inside first into selected element.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").after(" Online Web Development Tutorial"); }); }); </script> </head> <body> <p>Viral Polishwala</p>
 <button>Insert after the p element</button> </body> </html></pre>

- **before()** - Inserts content before the selected elements

Use	jQuery before() method add into start of selected elements.
Syntax	\$(selector).before(content)
Parameter	• Content: content is Required parameter. Insert content end into selected element.



Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").before(" Viral Polishwala"); }); }); </script> </head> <body> <p>Online Web Development Tutorial</p> <button>Insert after the p element</button> </body> </html></pre>
----------------	--

JQUERY REMOVE

To remove elements and content, there are mainly two jQuery methods:

- **remove()** - Removes the selected element (and its child elements)

Use	jQuery remove() method Removes all selected element content like child elements or text.
Syntax	\$(selector).remove()
Parameter	NA
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("div").remove(); }); }); </script> </head> <body> <div style="background:pink;">This is a paragraph.</div> <button>Click to empty p elements</button> </body> </html></pre>

- **empty()** - Removes the child elements from the selected element



Use	jQuery empty() method Removes all child elements from selected elements.
Syntax	\$(selector).empty()
Parameter	NA
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("div").empty(); }); }); </script> </head> <body> <div style="background: pink;">This is a paragraph.</div> <button>Click to empty p elements</button> </body> </html></pre>

CSS: Styling and Dimension

jQuery has several methods for CSS manipulation. We will look at the following methods:

- **addClass()** - Adds one or more classes to the selected elements

Use	jQuery CSS addClass() method add one or more CSS class to selected elements.
Syntax	\$(selector).addClass(classname)
Parameter	• ClassName: classname is Required parameter. add one or more css class names.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").addClass("param"); }); }); </script> <style type="text/css"> .param { font: 16px Verdana, Arial; font-weight: bold; color: pink; }</pre>



	<pre> } </style> </head> <body> <p>This is a first paragraph.</p> <p>This is a second paragraph.</p> <button>Add class into first P element</button> </body> </html></pre>
--	---

- **removeClass()** - Removes one or more classes from the selected elements

Use	jQuery removeClass() method removes one or all class from selected elements.
Syntax	\$(selector).removeClass(classname)
Parameter	• ClassName: classname is Required parameter. Remove one or more css class names.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").removeClass("param"); }); }); </script> <style type="text/css"> .param { font: 16px Verdana, Arial; font-weight:bold; color: pink; } </style> </head> <body> <p class="param">This is a first paragraph.</p> <p class="param">This is a second paragraph.</p> <button>Add class into first P element</button> </body> </html></pre>

- **toggleClass()** - Toggles between adding/removing classes from the selected elements

Use	jQuery toggleClass() method toggles add or remove one or more class from selected elements.
------------	---



Syntax	<code>\$(selector).toggleClass(classname)</code>
Parameter	• ClassName: classname is Required parameter. Replaces one or more css class names.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").toggleClass("param"); }); }); </script> <style type="text/css"> .param { font: 16px Verdana, Arial; font-weight:bold; color:pink; } </style> </head> <body> <p>This is a first paragraph.</p> <p>This is a second paragraph.</p> <button>ToggleClass add or remove class into P element</button> </body></html></pre>

- **css()** - Sets or returns the style attribute

Use	jQuery css() method set one or more style properties value into selected elements.
Syntax	<code>\$(selector).class(name,value) OR</code> <code>\$(selector).css({property:value,property:value, ...})</code>
Parameter	• Name: name is Required parameter. It is CSS property. • Value: value is required parameter. It is respective property value.
Example	<pre><html> <head> <script type="text/javascript" src="jquery.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").css("background-color","pink"); }); }); </script></pre>



<pre></head> <body> <p>This is CSS Refrence.</p> <button>Set the css properties value into p elements</button> </body> </html></pre>	
<pre><html> <head> <script type="text/javascript" src="jquery-1.8.0.min.js"></script> <script type="text/javascript"> \$(document).ready(function(){ \$("button").click(function(){ \$("p").css({"background-color":"pink","font-size":"20px"}); }); }); </script> </head> <body> <p>This is CSS Refrence.</p> <button>Set multiple css properties value into p elements</button> </body> </html></pre>	

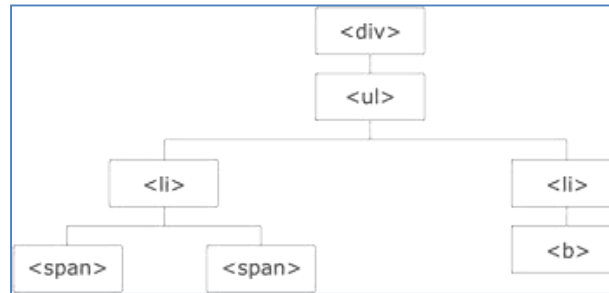
Traversing

jQuery is a very powerful tool which provides a variety of DOM traversal methods to help us select elements in a document randomly as well as in sequential method. jQuery traversing, which means "move through", are used to "find" (or select) HTML elements based on their relation to other elements. Start with one selection and move through that selection until you reach the elements you desire.

Most of the DOM Traversal Methods do not modify the jQuery object and they are used to filter out elements from a document based on given conditions.

```
<div>
  <ul>
    <li>list <span> item 1</span>and<span> item 2</span></li>
    <li>list <b>item 3</b></li>
  </ul>
</div>
```

The image below illustrates a family tree. With jQuery traversing, you can easily move up (ancestors), down (descendants) and sideways (siblings) in the family tree, starting from the selected (current) element. This movement is called traversing - or moving through - the DOM.



Explanation

- The `<div>` element is the parent of `<ul>`, and an ancestor of everything inside of it
- The `<ul>` element is the parent of both `<li>` elements, and a child of `<div>`
- The left `<li>` element is the parent of `<span>`, child of `<ul>` and a descendant of `<div>`
- The `<span>` element is a child of the left `<li>` and a descendant of `<ul>` and `<div>`
- The two `<li>` elements are siblings (they share the same parent)
- The right `<li>` element is the parent of `<b>`, child of `<ul>` and a descendant of `<div>`
- The `<b>` element is a child of the right `<li>` and a descendant of `<ul>` and `<div>`

ANCESTOR

An ancestor is a parent, grandparent, great-grandparent, and so on. With jQuery you can traverse up the DOM tree to find ancestors of an element.

Three useful jQuery methods for traversing up the DOM tree are: `parent( )`, `parents( )`, `parentsUntil( )`.

parent()

Use	The parent([selector]) method gets the direct parent of an element. If called on a set of elements, parent returns a set of their unique direct parent elements.
Syntax	selector.parent([selector])
Parameter	• selector – This is optional selector to filter the parent with.
Example	<pre><html> <head> <style> .ancestors * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px;</pre>



	<pre>} </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("span").parent().css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body> <div class="ancestors"> <div style="width:500px;">div (great-grandparent) ul (grandparent) li (direct parent) span </div> <div style="width:500px;">div (grandparent) <p>p (direct parent) span </p> </div> </div> </body> </html></pre>
	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = " jquery.js "></script> <script type = "text/javascript" language = "javascript"> \$(document).ready(function(){ \$("p").parent().addClass('highlight'); }); </script> <style> .highlight { background:yellow; } </style> </head> <body> <scan>Top Element</scan> <div></pre>



	<pre><div>sibling<div>child</div></div> <p>sibling</p> sibling </div> </body> </html></pre>
--	--

parents()

Use	The parents([selector]) method gets a set of elements containing the unique ancestors of the matched set of elements except for the root element.
Syntax	<i>selector</i> .parents([selector])
Parameter	selector – This is optional selector to filter the ancestors with.
Example	<pre><html> <head> <style> .ancestors * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("span").parents().css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body class="ancestors">body (great-great-grandparent) <div style="width:500px;">div (great-grandparent) ul (grandparent) li (direct parent) span </div> </body> <!-- The outer red border, before the body element, is the html element (also an ancestor) --> </html></pre>



	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = "jquery.js "></script> <script type = "text/javascript" language = "javascript"> \$(document).ready(function(){ var parentEls = \$("p").parents() map(function () { return this.tagName; }).get().join(", "); \$("b").append("" + parentEls + ""); }); </script> </head> <body> <scan>Top Element</scan> <div> <div class = "top">Top division <p class = "first">First Sibling</p> <scan>Second sibling</scan> <p class = "third">Third sibling</p> </div> Parents of <p> elements are: </div> </body> </html></pre>
--	--

parentsUntil()

Use	The parentsUntil() method returns all ancestor elements between two given arguments.
Syntax	selector.parentsUntil((selector))
Parameter	selector – Two selectors values need to be given between which the jQuery may fired



Example	<pre><html> <head> <style> .ancestors * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("span").parentsUntil("div").css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body class="ancestors"> body (great-great-grandparent) <div style="width:500px;">div (great-grandparent) ul (grandparent) li (direct parent) span </div> </body> </html></pre>
----------------	---

DESCENDANTS

A descendant is a child, grandchild, great-grandchild, and so on. With jQuery you can traverse down the DOM tree to find descendants of an element.

Two useful jQuery methods for traversing down the DOM tree are: `children()` and `find()`

`children()`

Use	The <code>children([selector])</code> method gets a set of elements containing all of the unique immediate children of each of the matched set of elements.
Syntax	<code>Selector.children([selector])</code>
Parameter	selector – This is an optional argument to filter out all the childrens. If not supplied then all the childrens are selected.



Example	<pre><html><head><style> .descendants * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("div").children().css({"color": "red", "border": "2px solid red"}); }); </script></head> <body> <div class="descendants" style="width:500px;">div (current element) <p>p (child) span (grandchild) </p> <p>p (child) span (grandchild) </p> </div> </body> </html></pre>
	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = "jquery.js"></script> <script> \$(document).ready(function(){ \$("div").children(".selected").addClass("blue"); }); </script> <style> .blue { color:blue; } </style> </head> <body> <div> Hello</pre>



	<pre><p class = "selected">Hello Again</p> <div class = "selected">And Again</div> <p class = "biggest">And One Last Time</p> </div> </body> </html></pre>
--	--

find()

Use	The find(selector) method searches for descendant elements that match the specified selector.
Syntax	<i>selector.find(selector)</i>
Parameter	selector – The selector can be written using CSS 1-3 selector syntax.
Example	<pre><html> <head> <style> .descendants * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("div").find("span").css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body> <div class="descendants" style="width:500px;">div (current element) <p>p (child) span (grandchild) </p> <p>p (child) span (grandchild) </p> </div> </body> </html></pre>



	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = "jquery.js"></script> <script type = "text/javascript" language = "javascript"> \$(document).ready(function() { \$("p").find("span").addClass("selected"); }); </script> <style> .selected { color:red; } </style> </head> <body> <div> <p>Hello, how are you?</p> <p>Me? I'm good.</p> </div> </body> </html></pre>
--	--

SIBLINGS

Siblings share the same parent. With jQuery you can traverse sideways in the DOM tree to find siblings of an element.

There are many useful jQuery methods for traversing sideways in the DOM tree: `siblings()` , `next()` , `nextAll()` , `nextUntil()` , `prev()` , `prevAll()` , `prevUntil()`)

siblings()

Use	The siblings([selector]) method gets a set of elements containing all of the unique siblings of each of the matched set of elements.
Syntax	<code>selector.siblings([selector])</code>
Parameter	selector – This is optional selector to filter the sibling Elements with.
Example	<pre><html> <head> <style> .siblings * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px;</pre>



	<pre>margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("h2").siblings().css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body class="siblings"> <div>div (parent) <p>p</p> span <h2>h2</h2> <h3>h3</h3> <p>p</p> </div> </body></html></pre>
	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = " jquery.js "></script> <script type = "text/javascript" language = "javascript"> \$(document).ready(function(){ \$("p").siblings('.selected').addClass("hilight"); }); </script> <style> .hilight { background:yellow; } </style> </head> <body> <div>Hello</div> <p class = "selected">Hello Again</p> <p>And Again</p> </body></html></pre>

next()

Use	The next([selector]) method gets a set of elements containing the unique next siblings of each of the given set of elements.
Syntax	<i>selector</i> .next([selector])



Parameter	• selector – The optional selector can be written using CSS 1-3 selector syntax. If we supply a selector expression, the element is unequivocally included as part of the object. If we do not supply one, the element would be tested for a match before it was included.
Example	<pre><html> <head> <style> .siblings * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("h2").next().css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body class="siblings"> <div>div (parent) <p>p</p> span <h2>h2</h2> <h3>h3</h3> <p>p</p> </div> </body> </html></pre>
	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = "jquery.js"></script> <script type = "text/javascript" language = "javascript"> \$(document).ready(function(){ \$("p").next(".selected").addClass("hilight"); }); </script> <style></pre>



	<pre>.highlight { background:yellow; } </style> </head> <body> <p>Hello</p> <p class = "selected">Hello Again</p> <div>And Again</div> </body> </html></pre>
--	---

nextAll()

Use	The nextAll([selector]) method finds all sibling elements after the current element.
Syntax	selector.nextAll([selector])
Parameter	selector – The optional selector can be written using CSS 1-3 selector syntax. If we supply a selector then result would be filtered out.
Example	<pre><html> <head> <style> .siblings * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src=" jquery.js "></script> <script> \$(document).ready(function(){ \$("h2").nextAll().css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body class="siblings"> <div>div (parent) <p>p</p> span <h2>h2</h2> <h3>h3</h3> <p>p</p> </div> </body></pre>



	<code></html></code>
	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = "jquery.js"></script> <script type = "text/javascript" language = "javascript"> \$(document).ready(function(){ \$("div:first").nextAll().addClass("highlight"); }); </script> <style> .highlight { background:yellow; } </style> </head> <body> <div>first</div> <div>sibling<div>child</div></div> <div>sibling</div> <div>sibling</div> </body> </html></pre>

nextUntil()

Use	The nextUntil() method returns all next sibling elements between two given arguments.
Syntax	<code>selector.nextUntil([selector])</code>
Parameter	selector – name of selector to start with and and work until the second selector.
Example	<pre><html> <head> <style> .siblings * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("h2").nextUntil("h6").css({"color": "red", "border": "2px solid red"});</pre>



	<pre>}); </script> </head> <body class="siblings"> <div>div (parent) <p>p</p> span <h2>h2</h2> <h3>h3</h3> <h4>h4</h4> <h5>h5</h5> <h6>h6</h6> <p>p</p> </div> </body> </html></pre>
--	---

prev()

Use	The prev([selector]) method gets the immediately preceding sibling of each element in the set of matched elements, optionally filtered by a selector.
Syntax	selector.prev([selector])
Parameter	selector – This is optional selector to filter the previous Elements with.
Example	<pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = "jquery.js"></script> <script type = "text/javascript" language = "javascript"> \$(document).ready(function(){ \$("p").prev(".selected").addClass("highlight"); }); </script> <style> .highlight { background:yellow; } </style> </head> <body> <div>Hello</div> <p class = "selected">Hello Again</p> <p>And Again</p> </body> </html></pre>

prevAll()



Use	The prev([selector]) method gets the immediately preceding sibling of each element in the set of matched elements, optionally filtered by a selector.
Syntax	<code>\$(selector).prevAll([filter])</code>
Parameter	• selector – This is optional selector to filter the previous Elements with. • Filter - Optional. Specifies a selector expression to narrow down the search for previous siblings
Example	<pre><html> <head> <style> .siblings * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$(".li.start").prevAll().css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body> <div style="width:500px;" class="siblings"> ul (parent) li (the previous sibling of li with class name "start") li (the previous sibling of li with class name "start") li (the previous sibling of li with class name "start") <li class="start">li (sibling with class name "start") li (sibling) li (sibling) </div> <p>In this example, we return all elements that are previous siblings of the li element with class name "start".</p> </body> </html></pre> <pre><html> <head> <title>The jQuery Example</title> <script type = "text/javascript" src = " jquery.js"></script></pre>



	<pre><script type = "text/javascript" language = "javascript"> \$(document).ready(function(){ \$("div:last").prevAll().addClass("highlight"); }); </script> <style> .highlight { background:yellow; } </style> </head> <body> <div class = "highlight">first</div> <div class = "highlight">sibling<div>child</div></div> <div class = "highlight">sibling</div> <div>sibling</div> </body> </html></pre>
--	---

prevUntil()

Use	The prevUntil() method returns all previous sibling elements between the selector and stop.
Syntax	\$(selector).prevUntil(stop,filter)
Parameter	• selector – This is optional selector to filter the previous Elements with. • Filter - Optional. Specifies a selector expression to narrow down the search for sibling elements between the selector and stop • Stop – Optional. A selector expression, element or jQuery object indicating where to stop the search for previous matching siblings elements
Example	<pre><html> <head> <style> .siblings * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("li.start").prevUntil("li.stop").css({"color": "red", "border": "2px solid red"}); }); </script></pre>



	<pre> </head> <body> <div style="width:500px;" class="siblings"> ul (parent) <li class="stop">li (sibling with class name "stop") li (the previous sibling of li with class name "start") li (the previous sibling of li with class name "start") li (the previous sibling of li with class name "start") <li class="start">li (sibling with class name "start") li (sibling) li (sibling) </div> <p>In this example, we return all previous sibling elements between the li element with class name "start" and the li element with class name "stop".</p> </body> </html> </pre>
--	---

FILTERING

The three most basic filtering methods are `first()`, `last()` and `eq()`, which allow you to select a specific element based on its position in a group of elements. Other filtering methods, like `filter()` and `not()` allow you to select elements that match, or do not match, a certain criteria.

first()

Use	The <code>first()</code> method returns the first element of the selected elements.
Syntax	<code>\$(selector).first()</code>
Parameter	NA
Example	<pre> <html> <head> <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.0/jquery.min.js"></script> <script> \$(document).ready(function(){ \$("div p").first().css("background-color", "yellow"); }); </script> </head> <body> <h1>Welcome to My Homepage</h1> <div style="border:1px solid black"> <p>This is a paragraph in a div.</p> <p>This is a paragraph in a div.</p> </pre>



	<pre></div>
 <div style="border:1px solid black"> <p>This is a paragraph in another div.</p> <p>This is a paragraph in another div.</p> </div> <p>This is also a paragraph.</p> </body></html></pre>
--	---

last()

Use	The last() method returns the last element of the selected elements.
Syntax	\$(selector).last()
Parameter	NA
Example	<pre><html> <head> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("div p").last().css("background-color", "yellow"); }); </script> </head> <body> <h1>Welcome to My Homepage</h1> <div style="border:1px solid black"> <p>This is a paragraph in a div.</p> <p>This is a paragraph in a div.</p> </div>
 <div style="border:1px solid black"> <p>This is a paragraph in another div.</p> <p>This is a paragraph in another div.</p> </div> <p>This is also a paragraph.</p> </body> </html></pre>

eq()

Use	The eq() method returns an element with a specific index number of the selected elements.
Syntax	\$(selector).eq(index)
Parameter	Index - Required. Specifies the index of the element. Can either be a positive or negative number.
Example	<pre><html> <head> <script src="jquery.js"></script></pre>



	<pre><script> \$(document).ready(function(){ \$("p").eq(1).css("background-color", "yellow"); }); </script> </head> <body> <h1>Welcome to My Homepage</h1> <p>My name is Donald (index 0).</p> <p>Donald Duck (index 1).</p> <p>I live in Duckburg (index 2).</p> <p>My best friend is Mickey (index 3).</p> </body> </html></pre>
--	--

not()

Use	The not() method returns elements that do not match a certain criteria. This method lets you specify a criteria. Elements that do not match the criteria are returned from the selection, and those that match will be removed. This method is often used to remove one or more elements from a group of selected elements.
Syntax	<code>\$(selector).not(criteria,function(index))</code>
Parameter	• criteria – Optional. Specifies a selector expression, a jQuery object or one or more elements to be removed from a group of selected elements. • Function(index)- Optional. Specifies a function to run for each element in a group. If it returns true, the element is removed. Otherwise, the element is kept.
Example	<pre><html> <head> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("p").not(".intro").css("background-color", "yellow"); }); </script> </head> <body> <h1>Welcome to My Homepage</h1> <p>My name is Donald.</p> <p class="intro">I live in Duckburg.</p> <p class="intro">I love Duckburg.</p> <p>My best friend is Mickey.</p> </body> </html></pre>



find()

Use	The find() method returns descendant elements of the selected element. A descendant is a child, grandchild, great-grandchild, and so on.
Syntax	<code>\$(selector).find(filter)</code>
Parameter	• Filter - Required. A selector expression, element or jQuery object to filter the search for descendants.
Example	<pre><html> <head> <style> .ancestors * { display: block; border: 2px solid lightgrey; color: lightgrey; padding: 5px; margin: 15px; } </style> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("ul").find("span").css({"color": "red", "border": "2px solid red"}); }); </script> </head> <body class="ancestors">body (great-grandparent) <div style="width:500px;">div (grandparent) ul (direct parent) li (child) span (grandchild) </div> </body></html></pre>

filter()

Use	The filter() method returns elements that match a certain criteria. This method lets you specify criteria. Elements that do not match the criteria are removed from the selection, and those that match will be returned. This method is often used to narrow down the search for an element in a group of selected elements.
Syntax	<code>\$(selector).filter(criteria,function(index))</code>
Parameter	• criteria – Optional. Specifies a selector expression, a jQuery object or one or more elements to be removed from a group of selected elements.



	• Function(index)- Optional. Specifies a function to run for each element in a group. If it returns true, the element is removed. Otherwise, the element is kept.
Example	<pre><html> <head> <script src="jquery.js"></script> <script> \$(document).ready(function(){ \$("p").filter(".intro").css("background-color", "yellow"); }); </script> </head> <body> <h1>Welcome to My Homepage</h1> <p>My name is Donald.</p> <p class="intro">I live in Duckburg.</p> <p class="intro">I love Duckburg.</p> <p>My best friend is Mickey.</p> </body> </html></pre>

